L'S
ELLIGENCE.

raordinary senses.
to superhuman
esight.

es a brilliant legal
rtner Foggy Nelson,
lawyer.



DYC0524606 C

# INDEX

NAME: Harisanjay S.S STD.: ____ SEC.: ____ ROLL NO.: 458 SUB.: Computer Networks Observation.

| S. No. | Date | Title | Page No. | Teacher's Sign / Remarks |
|---|---|---|---|---|
| 1. | 22/7/26 | Study of Network Commands | | 28/8/26 (9) |
| 2. | 28/07/26 | Study of different types of network cables. | | 4/8/26 (9) |
| 3. | 04/08/26 | Experiment on CISCO Packet Tracker. | (9) | 4/8/26 |
| 4. | 10/08/26 | Wireless LAN | (9) | 10/8 |
| 5. | 18/08/26 | Wireshark | (9) | 18/8 |
| 6. | 7/09/26 | Error detection & correction using Hamming code | (9) | 7/9 |
| 7. | 22/09/26 | Nmap to discover live host using Tryhackme platform | (9) | 22/9 |
| 8. | 28/09/26 | End to End communication at Transport layer | (9) | 28/9 |
| 9. | 28/9/26 | Sliding window protocol | (9) | 28/9 |
| 10 | 6/10/26 | Subnetting | (9) | 6/10 |
| 11. | 6/10/26. | ping program. | (9) | 6/10/26 |

| S. No. | Date | Title | Page No. | Teacher's Sign / Remarks |
|---|---|---|---|---|
| 12. | 12/10/26 | RAW Sockets to implement packet sniffing | 9 | 12/10/26 |
| 13. | 12/10/26 | Routing at Network Layer | 9 | 12/10/26 |
| 14. | 12/10/26 | Internetworking with routers | 9 | 12/10/26 |

22/7/26

Exp

Aim

used

arp-

resol

of

addres

hostn

comm

compu

ipconfig

Config

connec

and

on bts

with

Net B

net st

variety

TCP/IP

as wo

vstat's

| Page No. | Teacher's Sign / Remarks |
|---|---|
| 9 | 12/10/26 |
| 9 | 12/10/26 |
| 9 | 12/10/26 |

22/7/26

# EXP:1 STUDY OF NETWORK COMMANDS

Aim: To study of various network commands used in linux and windows.

arp-a:

ARP is short form of address resolution protocol, it will show the IP address of your computer along with the IP address and MAC address of your router

hostname:

This is the simplest of all TCP/IP commands it simply display the name of your computer.

ipconfig /all:

This command displays detailed configuration information about your TCP/IP connection including Router, Gateway, DNS, DHCP, and type of ethernet adapter in your system.

nbtstat-a:

This commands helps solve problems with Net BIOS name resolution. (NBT stands for Net BIOS over TCP/IP)

net stat:

(Network statistics) netstat displays a variety of statistics about a computers active TCP/IP connections both incoming and outgoing as well as viewing routing tables, interface statistic etc.

Nslookup:

(name server lookup) is a tool used to perform DNS lookups in Linux. It is used to display DNS details, such as the IP address of a particular computer, the MX records for a domain or the NS servers of a domain. nslookup can operate in two modus: interactive and non-interactive

Pathping:

Pathping is unique to window's and is basically a combination of the ping and tomands pathping traces the route to the destinating address than brunches as OS second list of each routes along the way gathering statistics on the rate of data loss along each hop.

Ping:

(Packet Internet Groper) command is the best way to test connectivity between two nodes. Ping use ICMP (Internet control messag Protocol) to communicate to other devices.

1.) #ping hostnames (ping localhost)
2.) #ping IP address (ping 4.2.2.2)
3.) #ping fully qualification domain name

Route:

Route command is used to show/ manipulate the IP routing table. It is primarily used to setup static routes to specific host or networks via an interface

Some impo

1.) IP

The IP comm
commands u
daily work.
and assignin
system. The
information,
network vari

IP (OPTIONS)

a.) To shoo
on your s
a

b.) To assig
OS
a.)
192.

c.) To delet
a.)
192

d.) After Th
The interfa
on

e.) After T
unabling t

promise o
g.) Ad
The local
reached

Some important linux networking commands

1.) IP

The ip commands is on of the basic commands every administrator will need in daily work. from setting up new systems and assigning IPs to troubleshooting existing system. The ip command can show address information, maniplate routing, plus display network various devices, interfaces and tunnels.

IP <OPTIONS> <OBJECT> <COMMANDS>

a.) To show the IP address to an interfac on your server

a.) [root@server ~] #ip address show

b.) To assign a IP to an interfac, for example OS

a.) [root@server ~] #ip address add 192.168.1.254/24 dev ups03

c.) To delete an IP on an interfac:

a.) [root@server ~] # ip address del 192.168.1.254/24 dev ups03

d.) After the status of the interfac by bringing the interfac into:

online: [root@server ~] #ip link set ens04

e.) Alter the status of the interfac by enabling promiscuous mode for

promisc on

g.) Add a default route (for all address) via the local gateway 192.168.1.254 that can be reached on devic.

h.) Add a route to 192.168.1.0/24 via the gateway at 192.168.1.254

[root@server ~] # ip route add 192.168.1.0/24 via 192.168.1.254

i.) Add a route for 192.168.1.0/24 via the gateway at that can be reached an device.

Who : [root@vserver ~] # ip route add 192.168.1.0/24 dev Who

j.) Delete the route for 192.168.1.0/24 via the gateway at 192.168.12.254:

[root@server ~] # ip route delete 192.168.1.0/24 via 192.168.1.254

k.) Display the route taken for IP 10.10.14:

[root IP@ server ~] # IP route get 10.10.14

2.) ifconfig

The ifconfig command was/is a staple in many sysadmin's tool belt for configuring and troubleshooting networks. It has since been replaced by the IP command discussed above.

3.) mtr

MTR (Matt's traceroute) is a program with a command-line interface that serves as a network diagnostic and trouble-shooting tool. The ping and traceroute commands. Just like a traceroute commands. will show the route from a computer to specified host with numerous of statistic

a.) The basic mtr command show you the statistic including each hop (hostnames) with time and loss :/. [root @ server ~] #mtr google.com

b.) show numeric IP address (if you use -g you will get IP address (numbers) instead of hostname :
[root @ server ~] mtr -g google.com.

c.) show the numeric IP address and hostname too : [root @ server ~] # mtr -b google.com.

d.) Set the number of pings that you want to send : [root @ server ~] # MTR -c 10 google.com

4.) tcpdump
The tcpdump command is designed for canturing and displaying packets.
You can install tcpdump with the command below : [root @ server ~] # dnf install -y tcpdump

Before starting any capture you need to know which interface can use you will need to used sudo or house root access in this case [root @ server ~] # tcpdump -D

If you want to capture traffic on who you can initiate that with tcpdump -i eth0 simple output : [root @ saver ~] tcpdump -i eth00

[root@server 2] # tcpdump -i eth0 -vv
capture traffic to and from one host
you can filter out traffic coming from
specific host. For example, to find traffic
coming from and going to 8.8.8.8 use
The command.
[root@server ~] # tcpdump -i eth0 -c 10 host
8.8.8.8
For traffic coming from 8.8.8.8 use
[root@server ~] # tcpdump -i eth0 src host
8.8.8.8
For outbound traffic going to 8.8.8.8 use
capture traffic to and from a
network you can also capture traffic to
and from a specific network using the
command below
[root@server ~]# tcpdump -i eth0 net
10.100 host 255.255.255.0
or
[root@server ~] # tcpdump -i eth0 0 port 53
For a specific host
[root@server ~] # tcpdump -i eth0 host
8.8.8.8 and port 53
To capture only HTTPS traffic
[root@server ~] # tcpdump -i eth0 -c10 host
www.google.com and port 443
To capture all port except port 80 and
25 [root@server ~] # tcpdump -i eth0 port
not 53 and not 25.
Result: The above both the windows
and linux commands are executed
and output are compiled successfully.

28/07/26 Exp
Aim:
cabl
a) Un
Diffe
Cable Type Categor
Categor
VTP Categor
Categor
STP Categor
SSP Categor

28/07/26

Exp: 2 → Study of different types of Network cables

Aim: Study of different types of Network cables

a.) Understand different types of Network cable

Different types of cables in Network cable.

1.) Unshielded Twisted Pair (UTP) cable
2.) Shielded Twisted Pair (STP) cable
3.) Co-axial cable
4.) Fibre optic cable.

| Cable Type | Category | Maximum Data Transmission | Advantages/ Disadvantages | Application/ Use |
|---|---|---|---|---|
| UTP | Category 3 | 10 bps | Advantages<br>* Cheaper in cost<br>* Easy to install as they have a smaller overall diameter<br>Disadvantages<br>*. More prone to (EMI) Electromagnetic interfan & noise. | 10Base-T Ethernet |
| | Category 5 | up to 100 Mbps | | Fast Ethernet<br>gigabit Ethernet |
| | Categoryse | 1 GB PS | | Fast Ethernet<br>Gigabit Ethernet |
| STP | Category 6,6a | 10 GbPS | Advantages<br>*. Shielded<br>* Faster than UTP<br>*. Less susceptible to noise and interference<br>Disadvantages<br>* Experience<br>*. Greater installation effort | Gigabit Ethernet<br>10 G Ethernet (55 m)<br>Widely used in data centres |
| SSP | Category 7 | 10 Gbps | | Gigabit Ethernet<br>10G Ethernet (100 m) |

| | | | | |
|---|---|---|---|---|
| Coaxial Cable | RG-6<br>RG-59<br>RG-11 | 10-100 Mbps | Advantage<br>* high bandwidth<br>* Immune to interfac<br>* low loss bandwidth<br>* versatile<br>Disadvantage<br>* limited distance<br>* cost<br>* size is bulky | Speed of sig<br>is 500 m<br>Television<br>network hig<br>speed inter<br>connections. |
| Fiber optics cable | single mode<br>multimode | 100 Gbps | Advantage<br>* high speed<br>* high bandwidth<br>* high security<br>* long distance<br>Disadvantage<br>* Expensive<br>* Request skilled installers | * Maximum<br>distance<br>fiber optic<br>cable is<br>around<br>100 meters |

Result: Thus, the different types of network cables are studied successfully

04/08/26

Exp-3 : E

Aim :

1.) Pur

Speed of signal is 500 m Television network high speed internet connections.

* Maximum distance of fibre optics cable is around 100 meters

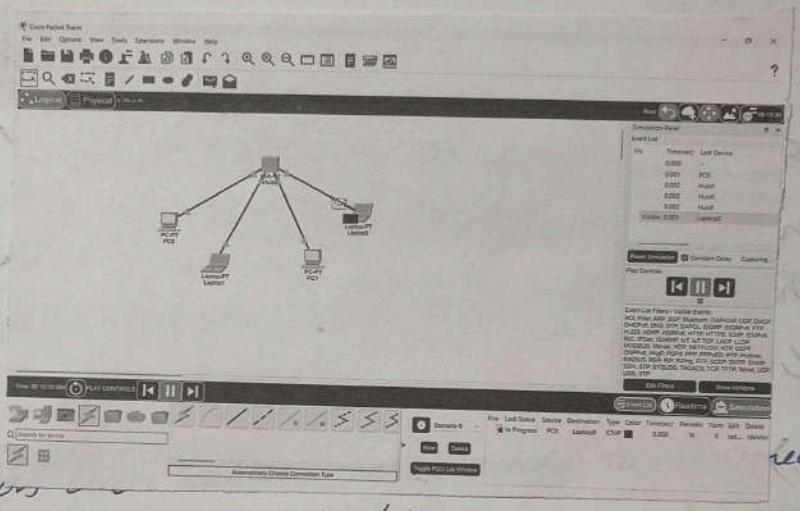
of successfully

04/08/26

# Exp-3 : Experiments on CISCO PACKET TRACKER (Simulation Tool)

Aim : To understand its environment and behaviour of network devices of CISCO packet tracker simulator.

## 1.) Pier to Pier Connection (P2P) :-

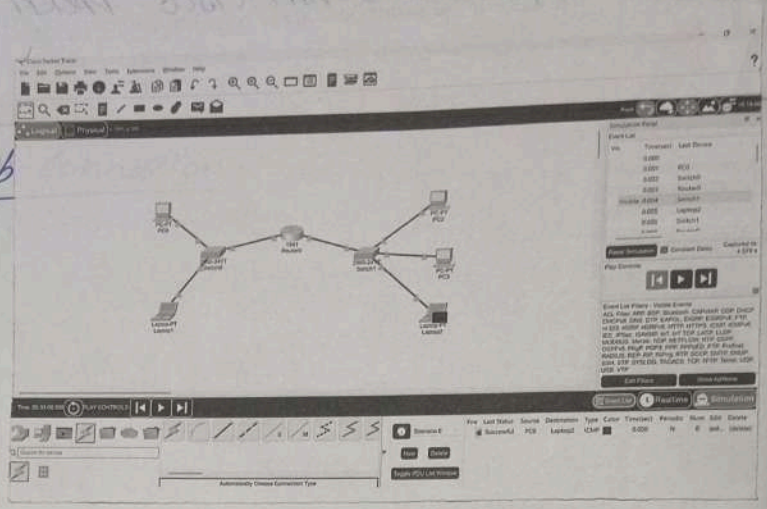


Cross [illegible] ect the PC and the laptop.

Only a cross-over cable can enable's communication directly between same type of End-Devices

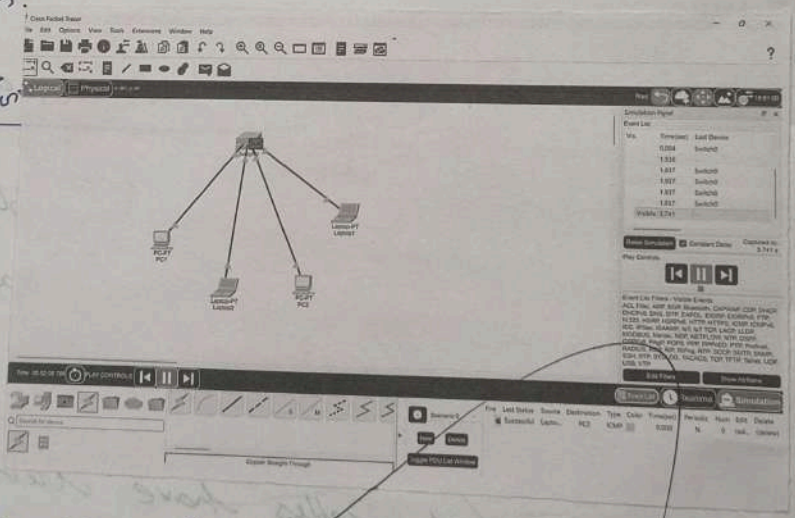
Usage of such cables have been replaced by switches, hubs and Routers

2.) Hub



A hub connects multiple devices in a network by blindly broadcasting all incoming data signals to every connected devices.

3.) Switch



Switch connects multiple devices within a single LAN and intelligently forwards data packets directly to specific destination using MAC addresses.

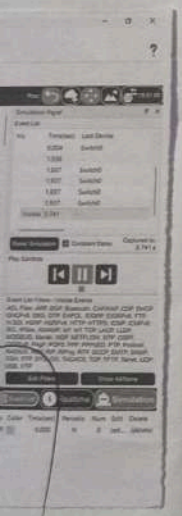
4.) R

A ro
distinct
traffic
default

Result:
PACKET
network
studie
success

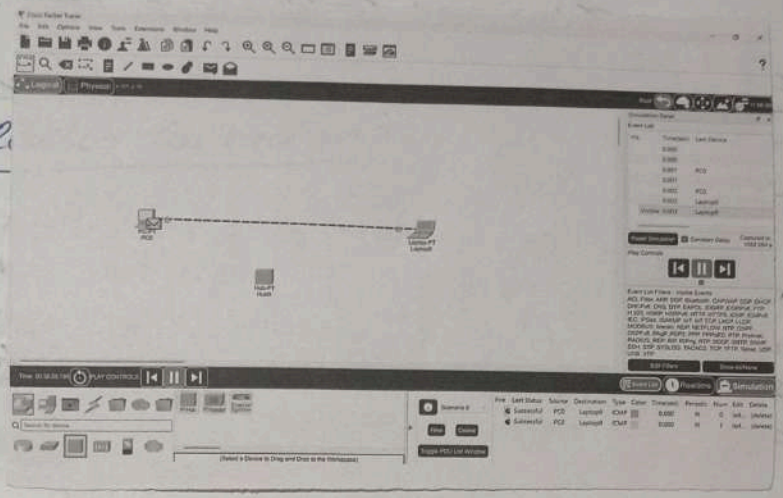


ious is a
all
connected



with this
words

4.) R



A router connects different subnets or distinct networks together, directing data traffic between them using IP addresses and default

Result: Hence the environment of CISCO PACKET TRACKER and behaviours of all the network devices in the software were studied, simulated and verified successfully.

9/8/24

10/08/24

Exp-4 Setup and Configure a LAN using Switch and Ethernet Cables.

What is LAN?

LAN stands for Local Area Network which is a group of computers and connected devices linked together in a small physical location like a home, school, or office building.

How to set up LAN

Step 1 :- Connect the Hardware

→ Place your router in a central location near a power outlet.

→ Connect your modem to the router's WAN or Internet port using an Ethernet cable.

→ Plug the router into power and wait for it to turn on.

Step 2: Connect your devices

→ wired setup :- plug one end of an Ethernet cable into your computer's network port and the other end into one of the numbered LAN ports on the router.

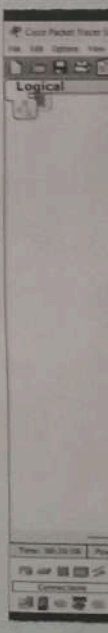
Step 3: Configure the router

→ Types your IP address (192.168.1.1 or 192.168.0.1) into the address bar and press Enter.

→ Log
and pa

Step 4:
and oth
comman
the de

Step 5:
from ot




Result:

10/8/24

LAN using

Cables.

Area Network

ers and

uthers in a

a home,

central

to the

using an

rowser and

end of an

puter's

end

ports

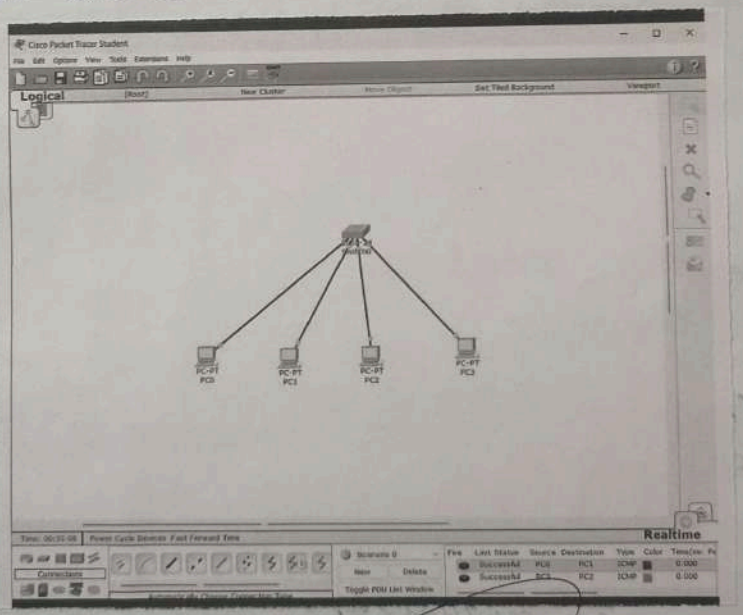
.168.1.1 on

bar and

-> Log in using the default username and password printed on the router.

Step 4: Check the connectivity between switch and other machine by using ping command in the command prompt of the device.

Step 5: Try to access the shared folder from others computers of the network




Result:

Wireless LAN using switch and ethernet cables are executed successfully in cisco packet tracer.

10/8/26

18/08/21

EXP : 5 WIRESHARK

AIM: Experiments on Packet capture tool: wireshark.

To filter, capture, view, packets in wireshark tool capture 100 packets from the Ethernet IEEE 802.3 LAN interface and save it.

Procedure:

* Select Local Area connection in wireshark
* To capture option
* Select stop capture automatically after 100 packets
* Then click start capture
* Search TCP packets in search bar
* To see flow graph click statistics flow graph
* Save the packets.

1.) Create a Filter to display only TCP/UDP packets, inspects the packets and provide the flow graph.

* Select LAN connection in wireshark
* To capture option
* Select stop capture automatically after 100 packets.
* Then click start capture.
* Save the packets

2.) Create a filter to display only ARP packets and inspect the packets

Procedure :-

* Go to capture option
* Select stop capture automatically after 100 packets
* Then click start capture
* Search ARP packets in search bar.
* Save the packets.

3.) Create a filter to display only HTTP packets and inspect the packets procedure.

* Select local area connection in wireshark.
* Go to capture option
* Select stop capture automatically after 100 packets
* Then click start capture.
* Search HTTP packets in search bar.

4.) Create a file to display only DNS packets and provide the flow graph.

Procedure :-

* Go to capture option.
* Select stop capture automatically after 100 packets.
* Then click start capture.
* Search DNS packets in search box.

5.) To create a filter to display only DHCP packets and inspect the packets.

* Select local area connection in Wireshark.
* Go to capture option
* Select stop capture automatically after 100 packets
* Then click start capture
* Search DHCP packets in search bar.
* Save the packets.

Result:

Wireshark on packet capture tool experiment are learned and executed successfully.

7/08/25

# Exp 6: Error detection & correction using Hamming code.

**Aim:**

Write a program to implement error detection and correction using hamming code concept. Make a test run to input data stream and verify error correction features.

**Sender:**

* Input to sender file should be a text of any length. Program should be convert the text to binary.
* Apply hamming code concept on the binary data and add redundant bits to it.
* Save this output in a file called channel.

**Code:-**

```
data = input("Enter 4 bits: ")
d1, d2, d3, d4 = map(int, data)
P1 = d1 ^ d2 ^ d4
P2 = d1 ^ d3 ^ d4
P4 = d2 ^ d3 ^ d4
code = [P1, P2, d1, P4, d2, d3, d4]
print("Sender Hamming code: ", "".join
      (map(str, code)))
```

Receiver side:-

* Receive the hamming code from the sender.
* Enter the received code.
* Calculate the parity check values.
* Find the error position using the parity values.
* Correct the error by changing the bit
* Display the corrected hamming code.

Code:-

```
r = list(map(int, input("Enter received 7 bits")))
r1 = r[0] ^ r[2] ^ r[4] ^ r[6]
r2 = r[1] ^ r[2] ^ r[5] ^ r[6]
r3 = r[3] ^ r[4] ^ r[5] ^ r[6]
error = r1 * 1 + r2 * 2 + r4 * 4
if error = 0:
    print("No Error")
else:
    print("Error at position:", error)
    r[error-1] ^= 1
    print("Corrected code:", "".join(map(
        str, received)))
```

Input:-

Enter 4 data bits: 1011
Sender Hamming code: 0110011
Received 7 bits: 0010011

Output:

Error
correct
Sample I
Enter
Sender
Enter re
Output :-
NO

Result:

Error

7/9/24 Hamming

Output:

Error at position : 2

corrected code : 0110011

Sample Input :-

Enter 4 data bits : 0010

Encoded Hamming code : 0101010

Enter received bits : 0101010

Output :-

No error.

Result:

Error detection & Error correction using Hamming code was executed successfully.

7/9/25

22/08/24

Exp 7: NMAP to Discover live hosts using Nmap scans on the TryHackMe platform

Aim:
To discover live hosts using Nmap vscans (ARP, ICMP, TCP/UDP) on the TryHackMe platform
Room link; https://tryhackme.com/room/nmap01

Procedure:

Task 1 :- Introduction

*. ARP vscan: This scan uses ARP requests to discover live hosts.

*. ICMP vscan: This vscan uses ICMP requests to identify live hosts

* TCP/UDP Ping scan: This vscan sends packets to TCP ports and UDP ports to determine like hosts

Task 2 :- Subnetworks

Network segment - It is a collection of computers linked together through a common "medium".

Subnetwork - It is usually the equivalence of one or more networks segments linked together and configured to utilize the same routing in an IP networks.

* Subnets with /16
* Subnets with /24

## Task 3 :- Enumerating Targets

We need to identify the targets we want to scan before we conduct any "action".

1.) List
- <MACHINE-IP> <scanme.nmap.org> <example>
- will scan 3 IP addresses.

2.) range
- 10.11.12.15-20
- will scan 6 IP addresses.

3.) Subnet
- <MACHINE-IP>/30
- will scan 4 IP addresses.

nmap -SL <TARGETS>

## Task 4 : Discovering live hosts.

1.) ARP from link layer - commonly used for Man-in-Middle-Attack (ARP Spoofing and ARP poisoning)

2.) ICMP from Network layer - commonly used in DDOS attacks.

3.) TCP from transport layer - SYN flooding, TCP reset, TCP session hijacking attacks.

4.) UDP from Transport layer - Also used in DDOS attacks.

2/10

## Task 5: Nmap - Host Discovery using ARP

1.) When a privileged user tries to scan LAN (local Area Network), Nmap uses ARP requests

2.) To discover internet hosts without host scanning. -live systems use nmap -sn Targets.

3.) nmap -PR -SN <TARGETS>
-PR specifies that you only want an ARP scan.

4.) nmap -PR -sn <MACHINE IP>/24

5.) nmap -PR -Sn <ATTACKBOX IP>/24

## Task6 :- Nmap host discovery using ICMP

1.) Add -PE option to utilize ICMP echo requests to discover live hosts

2.) nmap -PE -SN <TARGET to check host is live.

3.) -PP option instructs Nmap to use ICMP timestamp requests.
nmap -PP -SN TARGET

## Task 7:- Nmap host discovery using TCP and

1.) Nmap to use TCP SYN Ping use -PS followed by the port number, range, list of a combination of this options.

-PS21, -PS25 targets ports 21, 22, 23, 24, 25.

2.) nmap
is

3.) TCP
Nmap

4.) nmap
online

5.) UDP
open
response

6.) If
closed
an ICM

Task 8:-

1.) N
hosts

2.) -P
even for

Results:
NMAP

tryhackme

22/9/

2.) nmap -PS -SN TARGET check the TCP port is open.

3.) TCP ACK Ping - you must be running Nmap as a privileged user.

4.) nmap -PA -SN TARGET to discovered the online hosts on the target's subnet.

5.) UDP ping - sending a UDP packet to an open port is unlikely to result in a response.

6.) If we transmit a UDP packet to a closed UDP port, we should expect to receive an ICMP port unreachable packet.

nmap -PU -SN TARGET.

Task 8:- Using Reverse -DNS lookup

1.) Nmap is to use reverse -DNS interst hosts

2.) -R option to query the DNS server even for offline hosts.

Results:

NMAP to discover live host using tryhackme platform was successfully executed.

22/9/

28/09/26

# End to End communication at Transport layer.

Exp: 8

a.)

AIM:- Implement echo client server using TCP/UDP packets

Procedure:

* Create a socket using Python socket programming
* Specify the IP addresses and port number.
* Bind the server socket to the IP address and port.
* Make the server listen for client connections.
* Connect the client to the server.
* Send and Recive messages between client and server.
* Display the recived messag
* Close the ~~soc~~ socket connection.

Code:-

TCP - serve

```
import socket
server = socket.socket(socket.AF_INET,
socket SOCK_STREAM)
server.bind("127.0.0.1", 5000))
server.listen(1)
print("Server is waiting for connection")
conn, addr = server.accept()
print("Client connected").
```

```
while True
    data
if not data
    bre
    print(
    conn.
    conn.
server.close()
```

TCP_Receiver

```
import sock
client = soc
SOCK_STREA
client.conn
messag =
client.send
reply = cl
print(" se
client.clos
```

Terminal 1 -

Python tcp
server is w
connection
client conn
client: Hello

28/9/26 Result:

The T
server soc
successfully.

```
while True:
    data = conn.recv(1024).decode()
if not data:
    break
    print("client: ", data)
    conn.send(data.encode())
    conn.close()
server.close()
```

```
TCP_Receiver(client)
import socket
client = socket.socket(socket.AF_INET, socket.
SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
message = input("Enter message: "))
client.send(message.encode())
reply = client.recv(1024).decode()
print("server: ", reply)
client.close()
```

Terminal 1 - server

```
Python tcp_server.py
server is waiting for
connection...
client connected
client: Hello
```

Terminal 2 - client

```
Python tcp_client.py
Enter message: Hello
server: Hello
TCP echo is working
```

28/9/24

Result:

The TCP/UDP packets using echo client server socket program is executed successfully.

b) AIM: Implement chat client server using TCP packets

Chat-server side

```
import socket
server = socket.socket(socket.AF_INET, socket.
SOCKET_STREAM)
server.bind(("127.0.0.1", 60001))
server.listen()
print("Waiting for client....")
conn, addr = server.accept()
print("Client connected")
While True:
    message = conn.recv(1024).decode()
    if message.lower() == "bye":
        print("Client ended chat")
        break
    print("Client: ", message)
    reply = input("server: ")
    conn.send(reply.encode())
if reply.lower() == "bye";

conn.close()
server.close()
```

Chat-client side

```
import socket
client = socket.socket(socket.AF_INET, S
```

```
client.connect(("127.0.0.1", 6000))
print("Connected to server")
while True:
    message = input("Client: ")
    client.send(message.encode())
    if message.lower() == "bye":
        break
    reply = client.recv(1024).decode()
    print("Server: ", reply)
    if reply.lower() == "bye":
        break
client.close()
```

Terminal-1

```
python chat_server.py
Waiting for client....
Client connected
Client : Hello
Server : Hi
Client : How are you?
Server : Fine
```

Terminal-2

```
python chat_client.py
Connected to server
Client : Hello
Server : Hi
Client : How are you?
Server : Fine
```

Result:

The chat programming using sockets by TCP client & server was implemented successfully.

29/4

# Exp 9 : Sliding Window Protocol

**Aim:-**

Write a program to implement flow control at data link layer using sliding window protocol. Simulate the flows from one node to another.

**Code:-**

```
import time
window = int(input("Enter window size: "))
message = input("Enter message: ")
start = 0
while start < len(message)
    print("SENDER")
    end = start + window
    if end > len(message):
        end = len(message)

    #Sender Buffer
    sender_buffer = []
    for i in range(start, end):
        frame = (i, message[i])
        sender_buffer.append(frame)
        Print("Frame No:", i, "DATA:", message[i])
print("\n Sender Buffer: ")
print(Sender_Buffer)
# Delay
    print("\n Waiting for receiver..")
    time.sleep(1)
#Receiver
    print("Receiver")
    expected = start
    error = False
```

```
for frame in Sender_buffer:
    frame_no = frame[0]
    data = frame[1]
    print("Received Frame: ", frame_no, "DATA: ", data)
    if frame_no == expected:
        expected = expected + 1

else
    error = True
    break
#Reciver buffer
if error:
    recever_buffer = ("NACK", expected)
    print("\n Recevers Buffer: ")
    print(recever_buffer)

else
    ack = expected - 1
    recver_buffer = ("ACK", ack)
    print("\n Recver buffer: ")
    print(recver buffer")
# Sender reads ACK/NACK
print("\n SENDER CHECKING RESPONSE")
response = recver_buffer[0]
number = recver_buffer[1]
if response == "ACK":
    print("ACK received : ", number)
    start = number + 1

else print("NACK received: ", number)
    start = number
print("Transmission completed ");
```

OUTPUT:

Enter Window Size: 3
Enter message : Hello

SENDER

Frame No: 0 DATA: H
Frame No: 1 DATA: e
FRAME No: 2 DATA: l
Sender Buffer:
[(0, 'H'), (1, 'e'), (2, 'l')] waiting for recive....

waiting for receiver

RECEIVER

Recived Frame: 0 DATA: H
Recived Frame: 1 DATA: e
Recived Frame: 2 DATA: l
Recvd Buffer:
('ACK', 2)

SENDER CHECKING RESPONSE
Ack Received: 2

SENDER

Frame No: 3 DATA: l
Frame No: 4 DATA: o
Sender Buffer:
[(3, 'l'), (4, 'o')]

RECEIVER

Recived Frame: 3 DATA: l
Recived Frame: 4 DATA: o
Recver Buffer:
('ACK', 4)

SENDER CHECKING RESPONSE
ACK Recived: 4

TRANSMISSION COMPLETED

RESULT:

Sliding window protocols are learned and executed successfully

Exp10 : Subnetting in Cisco Packet Tracer

Aim:
To implement classless IP subnetting using cisco packet tracer and verify the connecting connectivity between different subnets.

Requirements
* Cisco Packet Tracer
* Routers
* Switches
* PCs
* Ethernet Cables

Procedure
1.) Open Cisco Packet Tracer
2.) Create a new network topology
3.) Add routers, switches, and PC's to the workspace. Connect the device using ethernet cables.
4.) Divide the network into smaller subnets using the 127 subnet mask.
5.) Configure the IP address and subnet mask of the router interfaces
6.) Configure the IP address, subnet mask and default gateway for each PC.
7.) Configure the required switch ports
8) Enable the router interfaces using the shutdown command.
9.) Check the IP configuration of all devices. Use the ping command to check connectivity between PCs.
10.) Verify that the devices in the network communicate successfully.

EXP 11

AIM:
To ...
programm...
two net...

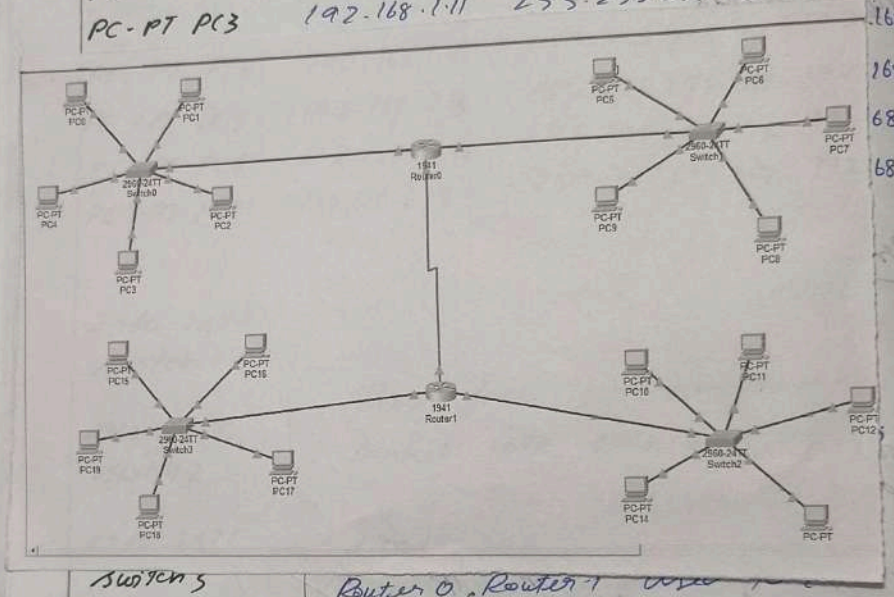
Program:
impo...
impo...
impo...
def ...

def ...

try:

IP addressing

| | IP address | Subnet Mask | Default Gateway |
|---|---|---|---|
| PC-PT PC0 | 192.168.2.12 | 255.255.255.192 | 192.168.3.1 |
| PC-PT PC1 | 192.168.2.11 | 255.255.255.192 | 192.168.3.1 |
| PC-PT PC2 | 192.168.1.12 | 255.255.255.192 | 192.168.1.1 |
| PC-PT PC3 | 192.168.1.11 | 255.255.255.192 | 192.168.1.1 |




168.4.1
68.2.1
68-2-1

Switch 3

2960-24TT
switch4

Router 0, Router 1 ... communicate routers to routes and also switches.

Result:

Thus, classless IP subnetting was successfully implemented using cisco packet tracker, and connectivity between the network devices was verified successfully

Mask | Default Gateway

.255.192 | 192.168.3.1
.255.192 | 192.168.3.1
.255.192 | 192.168.[illegible].1
255.192 | 192.168.[illegible].1




s to routes
s.

was
s10 packet
the network

EXP 11 PING PROGRAM

AIM:
To implement a ping program using socket programming to check the connectivity between two network devices.

Program:

```
import socket
import time
import sys
def udp_ping_server(host="127.0.0.1", port=12000):
    server_socket = socket.socket(socket.AF_INET,
                                  SOCK_DGRAM)
    server_socket.bind((host, port))
    print(f"[server] listening on {host}:{port}")
    while True:
        msg, client_address = server_socket.recvfrom
                                  (1024)
        print(f"[SERVER] Received '{msg.decode()}'
              from {client_address}")
        server_socket.sendto(msg client_Address)
def udp_ping_client(server_host = "127.0.0.1",
                    server_port, 12000,
    Count = 5);
    client_socket.settimeout()
    for i in range(1, count+1):
        msg = f"ping {i} {time.time()}"
        start = time.time()
        client_socket.sendto()
            msg.encode(), (server_host, server_
                           port))
try:
    data = client_socket.recvfrom(1024)
    end = time.time()
    rtt = (end-start) * 1000
```

```
print (
    f"Reply from {server_host}: {server_port}
    f"{data.decode()} RTT = {rtt:.2f} ms")
except socket.timeout:
    print(f"Request {i} timed out")
client_socket.close()
if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "server":
        udp_ping_server()
    else:
        udp_ping_client()
```

Output:

```
python udp_ping.py server
[SERVER] Listening on 127.0.0.1:12000
[SERVER] Received 'ping 1 1791476483.656088'
        from ('127.0.0.1', 58522)
[SERVER] Received 'ping 2 1791476483.8980255'
        from ('127.0.0.1', 58522)
python udp_ping.py
  Reply from 127.0.0.1:12000 ping 1 1791476483 RTT
  = 0.92 ms
  Reply from 127.0.0.1:12000 ping 2 1791476483 RTT
  = 0.25 ms
```

RESULT:

Thus The ping program is successfully executed.

12/10/26

12/10/26

Expt2 :

AIM:-

To ...
sockets ...
packets.

Algorithm

* . Start
* . Creat... SOCKET
* . Captu...
* . Extrac... each
* . Identi... destinati... protocol.
* . Displ...
* . Repea... Stop...

CODE:

from ...
from ...
dy pack...
if I...
...
...
...
# ...

12/10/26

## Exp12: Write a code using RAW Sockets to implement packet sniffing

**AIM:**

To write a python program using RAW sockets to capture and analyze network packets.

**Algorithm:**

* Start the program
* Create a raw socket using SOCKET.AF_PACKET and SOCKET.SOCK_RAW.
* Capture incoming network packets
* Extract the Ethernet header from each packet.
* Identify the source MAC address, destination MAC address and ethernet protocol.
* Display the packet details.
* Repeat until the program is stopped.

**CODE:**

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP
def packet_callback(packet):
    if IP in PACKET:
        ip_layer = PACKET[IP]
        Protocol = IP_layer.Proto
        src_ip = ip_layer.src
        dest_ip = ip_layer.dest
    # determine the protocol
        Protocol_name = " "
        if Protocol == 1:
            Protocol = name = "ICMP"
        elif Protocol == 6:
            Protocol_name = "TCP"
```

```
elif protocol == 17:
    Protocol_name = "UDP"
else:
    Protocol_name = "unknown protocol"
print(f"Protocol: {protocol_name}")
print(f"Source IP: {src_ip}")
print(f"Destination IP: {dest_ip}")
print("-" * 50)

sniff(iface="wifi", prn=packet_callback,
      filter="ip", store=0)
```

OUTPUT:

```
Destination IP: 224.0.0.251
Protocol: TCP
Source IP: 172.16.11.15
Destination IP: 172.16.8.1
Protocol: UDP
Source IP: 172.16.11.136
```

```
Router>enable
Router#show ip route static
S    30.0.0.0/8 [20/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router#show ip route static
S    30.0.0.0/8 [10/0] via 20.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2
```

Result: Thus the python [illegible] successfully executed to check the socket programming messages were successfully sent and received between client and server.

12/10/26

12/10/26

Exp 13

a) Aim

config

AIM

To

config

PROCED

*

Creat

*

Switc

*

PC's

*

Shuto

*

Router

*

Differ

*

*

*

*

That

*

-ful

12/10/26

## Exp 13: Routing at Network Layer

a) Simulate static routing protocol configuration using Cisco packet Tracer.

AIM:

To simulate a static routing configuration using CISCO PACKET TRACER.

PROCEDURE:

* Open Cisco Packet Tracer and create the given network topology
* Connect 3 routers and PC's using suitable cables.
* Configure IP addresses for routers and PC's
* Enable all router interfaces using no shutdown.
* Configure static routers on Router 0, Router 1, and Router 2 using IP mobile.
* Set main and backup router using different administrative Distance value.
* Test Connectivity
* Use Tracer to verify the routing path.
* Disconnect the main link and verify that the backup route works.
* Thus, a static routing is successfully configured and verified.

```
Pinging 20.0.0.2 with 32 bytes of data:

Reply from 20.0.0.2: bytes=32 time=55ms TTL=125
Reply from 20.0.0.2: bytes=32 time=56ms TTL=125
Reply from 20.0.0.2: bytes=32 time=60ms TTL=125
Reply from 20.0.0.2: bytes=32 time=55ms TTL=125

Ping statistics for 20.0.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 55ms, Maximum = 60ms, Average = 56ms

C:\>tracert 20.0.0.2

Tracing route to 20.0.0.2 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    11 ms     8 ms      5 ms      192.168.1.250
  3    29 ms     26 ms     25 ms     192.168.1.245
  4    22 ms     28 ms     20 ms     20.0.0.2

Trace complete.
```

Result :

Thus, static routing was successfully configured in CISCO PACKET TRACER. Connectivity between all networks was verified successfully using Ping.

b.) Simulat

AIM:-
To si
TRACER.

PROCEDUR
* · Creat
2 PCs
* · Connect
* · Assign
* · Config
interfac
* · Enabl
* · Set
DE seri
* · Enable
RIP.
* · Add
The net
* · Test
* · Use
selected
* · Disconn
Link an
RIP selec

b) Simulate RIP using CISCO PACKET TRACER.

AIM:-
To simulate RIP using CISCO PACKET TRACER.

PROCEDURE:
* Create the topology with routers and 2PCs
* Connect the device using suitable cables.
* Assign the given IP address to both PCs.
* Configure Fast ethernet and serial interface of all routers.
* Enable interface using no shutdown.
* Set clock rate & bandwidth 64 on DE serial interface.
* Enable RIP on all routers using router RIP.
* Add directly connected networks using the network command.
* Test connectivity using ping 20.0.0.2
* Use tracert 20.0.0.2 to verify the selected route.
* Disconnect the direct Router 0_Router 2 link and use tracert again to verify RIP selects the alternate route.

as successfully.
ACER. Connectivity
verified

12/10/26

Exp 14 :

a.) Design an internetwork

Aim :

To design internetwork PACKET TRACER two PCs.

PROCEDURE :

Step 1 : Conf

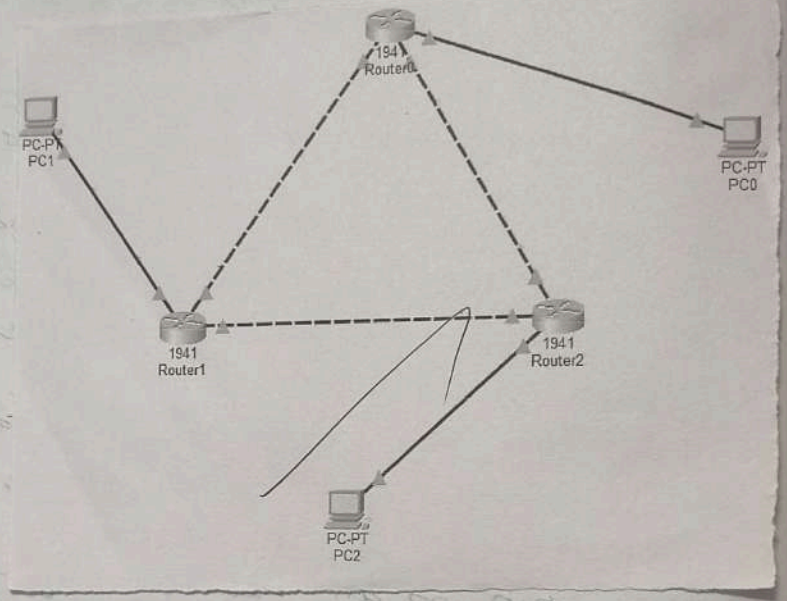
* Select the
* Press en
* Type enab mode :

Step 2 :

* Assign IP network.
* Select the IP configu address, Defa
* Assign the 192.168.10
* Assign th 192.168.2.01

Step 3 :

* Connect fas fast ethern via copper




Result :

12/10/26

RIP successfully configured and dynamic routing between PC0 and PC1 was verified

12/10/26

# EXP 14: Internetworking with routers in CISCO PACKET TRACER Simulator.

a.) Design and configure a simple internetwork using a router.

Aim:

To design and configure a simple internetwork using a router in CISCO PACKET TRACER and fast connectivity between two PCs.

PROCEDURE:

Step 1: Configuring Router 1

* Select the routers and open [1].
* Press enter to start configuring Router1.
* Type enable to activate the privileged mode:

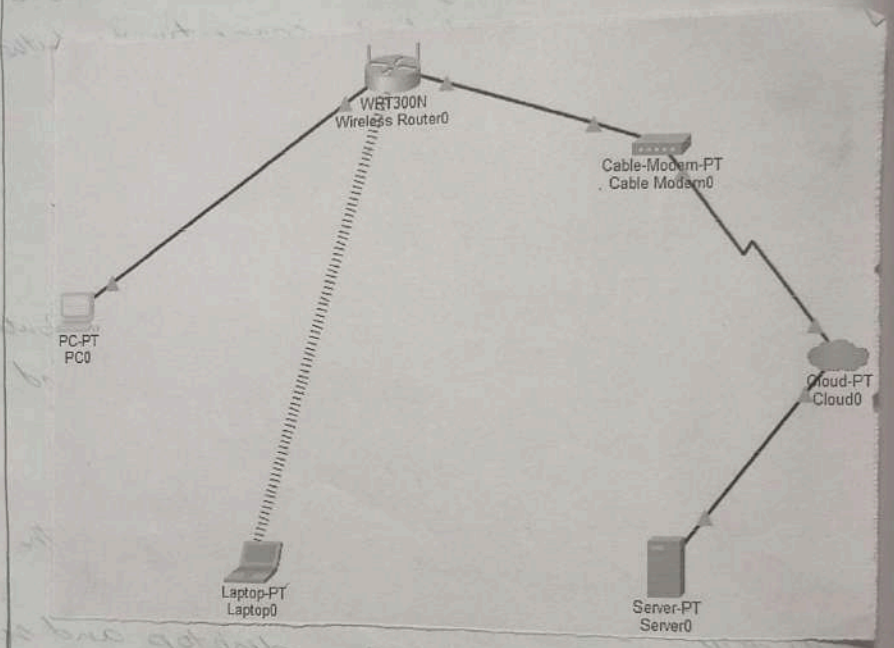
Step 2:

* Assign IP addresses to every PC in the network.
* Select the PC, Go to the desktop and select IP configuration and assign an IP address, Default gateway subnet mask.
* Assign the default gateway of PC0 as 192.168.10.1
* Assign the default gateway of PC1 as 192.168.2.01.

Step 3:

* Connect fast Ethernet 0 port of PC0 with fast ethernet 0/0 port of Router 1 using a copper straight-through cable.

* Connect Fast Ethernet 0 port of PC1 with fast ethernet 0/1 port of Router1 using a copper straight through cable.




Result: The simple internetwork was successfully designed and configured using a router connectivity between PC0 and PC1 was successfully verified using PDU.

12/10/22

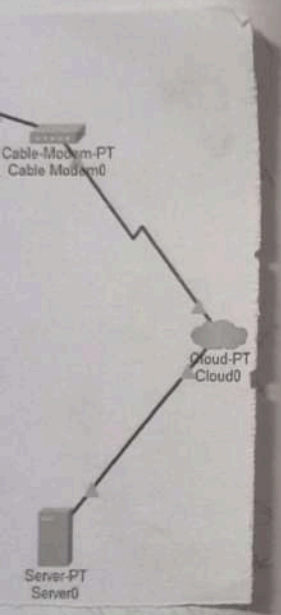
b) Design
using
cloud.

Aim:-
using a
interval

PROCEDURE

* Open
requir
* Connect
modem,
* Configu
home ne
* Connect
network
* Configu
and ena
* Verify
Cisco co

port of PC with
Router, using a
it.




etwork was
onfigured using
en PCo and PU
ng PDU.

b) Design and configure an internet network using wireless router DHCP and ~ ter cloud.

Aim:- To design and configure an internetwork using a wireless router, DHCP server and internet cloud in CISCO PACKET TRACER.

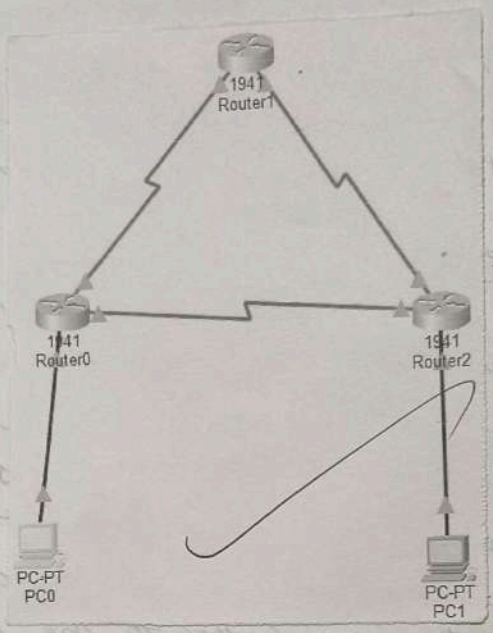
PROCEDURE:-

* Open Cisco Packet Tracer and create the required network topology.
* Connect PC, laptop, wireless router, cable modem, internet cloud and server.
* Configure the wireless router with SSID home network and enable DHCP
* Connect the two laptop to the wireless network and set the PC to DHCP.
* Configure the server with IP 208.67.220.220 and enable DHCP and DNS.
* Verify connectivity using ip config and ping Cisco.com.

c.) Design and
in your
and Ethe

Aim:-
To desig
internetwork
ethernet cab
computers.

Procedure:
* Open Cisco
a switch
* Connect
ethernet ca
* Connect
Ethernet co
* Assign
and defau
computers.
* Configur
appropriat
interfaces.
* Use the
network
* Observe th
packets.




Result:
The internetwork was successfully designd and configured using a wireless router, DHCP server and internet cloud and connectivity was successfully verified using Ping Cisco.com.

12/10/22

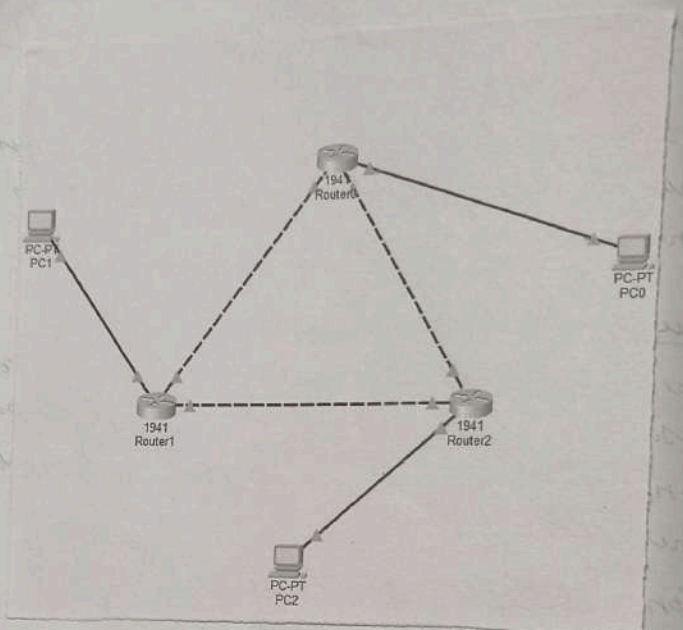
c.) Design and configure an internetwork in your lab using Switch, router and Ethernet cables.

Aim:-

To design and configure an internetwork using a switch, router and ethernet cables and verify connectivity between computers.

Procedure:

* Open Cisco packet Tracer and place computers, a switch and router in the workspace.
* Connect the computers to the switch using ethernet cables.
* Connect the switch to the router using an Ethernet cable.
* Assign Static IP addresses, subnet masks and default gateway addresses to the computers.
* Configure the router interfaces with appropriate IP address and enable the interfaces.
* Use the ping command to test connectivity between computers
* Observe the successful transmission of packets.




Result: Thus an internetwork was successfully designed and configured using a switch, router and ethernet cables in cisco packet Tracer.